PROJECT INTERVIEW PREPARATION

Etihad Travel Assistant
100 speaker-ready answers

A companion to the 100-question guide. Each response is phrased in the first person so you can explain the engineering decision, the implementation, and its limits clearly to a client interviewer.

HOW TO PRACTISE

01 Start with the user problem and the decision you made.

02 Name the component that implements it, then give one concrete example.

03 Distinguish what is built today from what you would improve next.

Based on the local project code reviewed 11 October 2026.

This is interview practice, not an official Etihad policy publication.

Project and architecture

001

What user problem does the Etihad Travel Assistant solve, and which requests are deliberately outside its scope?

I built this as a policy-grounded travel assistant. It answers Etihad policy questions with source citations and sends live flight questions to AviationStack. It does not book or change tickets, decide eligibility for refunds, or pretend to have account access. When the available evidence is insufficient, it says so rather than inventing a result.

002

How does a request move from the Streamlit chat input to the FastAPI response and back to the user?

The user types in Streamlit, which sends an authenticated request to FastAPI. The backend loads that user's conversation, invokes the LangGraph workflow, and returns an answer, decision, state, and citations. Streamlit renders those fields, while the backend records the conversation and audit event. The UI is a client; business rules live in the backend.

003

Why did you choose FastAPI for the backend and Streamlit for the user interface in this local MVP?

FastAPI gives me typed request validation, authentication dependencies, and clear endpoints. Streamlit let me build and iterate on a chat and admin dashboard quickly. That combination is appropriate for a local MVP, although I would choose a more controlled frontend and deployment architecture if the product had to serve a large public audience.

004

How does the application distinguish a policy question from a live flight-information request?

I route by intent before doing retrieval. Flight-status, schedules, routes, first-flight, and arrivals requests go to the AviationStack MCP branch; policy requests go through cache lookup and RAG. That separation matters because a static policy document is not a trustworthy source for a flight's current operational status.

Project and architecture (continued)

005

What responsibilities do LangChain, LangGraph, and LangSmith each have in this project?

LangChain provides the OpenAI chat and embedding integrations and the MCP adapter. LangGraph defines the conditional workflow and preserves state across clarification turns. LangSmith is optional observability for traces; it is not the database and does not make an extra model call merely by being enabled.

006

Which components own policy data, live flight data, conversation history, and reusable answers?

The scraped policy JSON is the source corpus, and Chroma stores its reusable vectors; BM25 ranks the same policy chunks lexically. AviationStack is the live flight source through MCP. SQLite stores users, conversations, messages, feedback, and LLM usage. Redis stores reusable generic policy answers, not chat history or flight status.

007

What is the complete path of a first-time baggage-policy question, including retrieval, refinement, citations, and caching?

For a first-time baggage question, I first check the shared Redis key. On a miss, BM25 and Chroma retrieve policy candidates, the graph selects question-relevant excerpts, and OpenAI refines a concise answer from those excerpts. The response carries source IDs and citations. A successful generic answer is then cached for later users.

008

What is the complete path of a flight-status question when the user initially omits the departure date?

The graph detects a flight-status intent but sees that the date is missing, so it asks for YYYY-MM-DD and saves the pending intent and flight number in conversation state. When the date arrives, the next turn fills that slot and calls the flight MCP tool. The response uses AviationStack's normalized status, or reports that no verified status was found.

Project and architecture (continued)

009

Which parts of this project are deterministic, and which parts depend on a language model or an external API?

Authentication, role checks, intent routing, cache lookup, retrieval ranking, status validation, and refusal rules are code-driven. OpenAI is used to refine policy evidence, summarize valid MCP data, or answer narrowly allowed general travel advice. AviationStack supplies live flight records. Keeping these roles explicit makes the system easier to test and audit.

010

What trade-offs did you accept by building a local MVP rather than a production deployment?

I optimized this for proving the workflow locally: FastAPI, Streamlit, SQLite, a local Chroma index, and optional Redis Cloud. The trade-off is that this is not a hardened public service: it still needs stronger secret management, deployment, monitoring, access controls, and automated source refresh before production use.

Policy ingestion and sources

011

How is the Etihad Hub policy library collected, and what completeness checks prevent a partial scrape?

The ingestion command fetches the public Etihad Hub policy page and parses its five policy-library groups. It checks that all five expected groups are present; if the page structure changes or a group is missing, ingestion fails instead of silently publishing a partial corpus. The current snapshot is then transformed into document records and retrieval chunks.

012

Why does the project preserve original HTML, normalized text, source links, and scrape metadata?

I keep the original HTML and normalized text so I can re-check how a passage was interpreted. Links, document titles, section paths, scrape time, and source hash support provenance and troubleshooting. If a user challenges an answer, I can trace it to the policy page rather than relying on an unexplained model statement.

013

How are policy documents divided into retrieval chunks, and what can go wrong if chunks are too large or too small?

The splitter tries to preserve paragraph boundaries, targeting roughly 1,400 characters with a small overlap for continuity. Huge chunks mix unrelated rules and waste context; tiny chunks can separate a condition from its exception. I would validate chunking against real questions, especially tables and policies whose meaning depends on headings.

014

How does the system keep supplemental official Economy baggage information separate from the Hub snapshot?

The Economy baggage details live in a separate official-source JSON supplement instead of being blended invisibly into the Hub scrape. Each chunk still has its own title, URL, and version metadata. That separation makes it clear which answer came from the Hub snapshot and which came from another official Etihad page.

Policy ingestion and sources (continued)

015

What does the corpus digest represent, and how is it useful when the underlying policy documents change?

The corpus digest is a SHA-256 fingerprint of the main corpus plus supplements. I use it to give updated content a different cache identity, so answers based on old text are not automatically reused. The Chroma collection also incorporates the corpus fingerprint and embedding model, allowing changed policy content to be indexed separately.

016

How would you detect that a scraped policy is outdated, superseded, or effective only on a future date?

The current MVP records scrape time and source metadata, but that alone does not prove a policy is still current. I would compare live page hashes on a schedule, review effective-date language, and mark superseded chunks inactive. Until that automation exists, I would tell users the source is a dated snapshot and re-check time-sensitive policy details.

017

What would you do if two official Etihad pages appeared to disagree on the same allowance?

I would not merge conflicting allowances into one confident statement. I would retain both sources and their dates, determine which fare, route, and effective period each applies to, and ask for missing context. If the conflict remains, the assistant should explicitly flag it and direct the traveler to Etihad rather than choose an unsupported number.

018

How do you preserve section paths and citations so a retrieved answer can be traced to its source?

Each policy chunk carries an ID, document title, section path, version, passage, and source URL. Retrieved IDs travel with the generated claim, and the service turns them into citations in the response. That creates a practical audit trail from a sentence in the answer back to the supporting policy excerpt.

019

What tests would confirm that a refresh has not silently dropped a policy group or important table?

I would test the expected five groups, document and chunk counts, stable IDs, source URLs, and representative table rows after every refresh. I would also run known policy questions against the new corpus and compare retrieved source IDs with a golden set. A large unexpected count change would fail review rather than publish automatically.

Policy ingestion and sources (continued)

020

How would you handle a page redesign that changes the scraper's expected HTML structure?

The scraper is intentionally strict: if its expected page structure disappears, it raises an error. I would inspect the new HTML, update the parser and tests, then compare the new output with the last known good snapshot. I would not silently fall back to a possibly incomplete policy set.

Hybrid search and Chroma

021

How does BM25 rank policy chunks, and which types of exact-match questions does it handle well?

BM25 scores policy chunks by how strongly their words match the query, with adjustments for term frequency and document length. In this project, title overlap adds extra weight. It is especially useful for exact fare names, policy labels, and quantities that semantic similarity might treat too loosely.

022

How are OpenAI embeddings and persistent Chroma collections used for semantic policy retrieval?

For semantic search, the project embeds each current policy chunk with OpenAI and stores the vectors in a persistent Chroma collection. It embeds the user's policy question and asks Chroma for nearby vectors. Persistence means the corpus does not need to be embedded again on every restart, unless content or the embedding model changes.

023

Why is combining BM25 and semantic search preferable to using either method alone for this corpus?

The two methods solve different retrieval failures. BM25 protects precise terms such as Economy Value or a policy title, while semantic search can recover a paraphrase such as 'bags I can take' for 'baggage allowance.' I combine them, then still require answer evidence; I do not treat a high vector rank as proof by itself.

024

What was the effect of the original BM25-first early return, and how does the current retriever avoid it?

Originally, the Chroma retriever returned immediately whenever BM25 found nothing. That made semantic search powerless for the very paraphrases it was meant to handle. I removed that gate, so both searches run independently and their candidates are fused. A regression test now covers the BM25-miss case.

025

How are lexical and vector candidates combined, and what is the purpose of rank fusion?

The retriever adds rank-based points from each list, with stronger weight on lexical ranking and a separate contribution from vector ranking. The merged top results become candidates for evidence selection. I chose this simple, inspectable fusion for the MVP; I would tune its weights with a labeled evaluation set rather than assume they are universally optimal.

Hybrid search and Chroma (continued)

026

When might a semantic search return a related but incorrect policy, and how should that result be filtered?

A vector can find a policy about a related topic but the wrong fare, date, or passenger type. My downstream evidence selector looks for question-matching passages, and the prompt tells the model to answer only from those excerpts. I would add calibrated relevance thresholds and policy-scope filters once I have enough labeled examples to measure false positives.

027

How would you tune the number of retrieved chunks without overwhelming the answer-refinement step?

Today the retriever returns a small final set, and the evidence selector sends at most a few short passages to the model. I would tune top-k against a golden set: too low hurts recall, while too high adds irrelevant context, latency, and cost. I would track both retrieval hit rate and final answer faithfulness, not just one metric.

028

How does Chroma reuse an existing policy index after an application restart, and when must it re-embed?

The collection name depends on the corpus digest and embedding model. At startup, Chroma opens the persistent local directory; on first use, the repository checks which current chunk IDs are already present and embeds only missing ones. A corpus or embedding-model change leads to a new collection identity and appropriate reindexing.

029

What should happen if Chroma or the embedding API fails while BM25 still has a useful match?

The behavior is designed to degrade honestly. If Chroma cannot initialize, the service uses local BM25. If an embedding query fails, the retriever returns the lexical candidates and records a vector warning. If neither retrieval path supplies useful evidence, the policy branch states that it cannot verify the answer; it does not improvise.

Hybrid search and Chroma (continued)

030

Which evaluation questions would you use to compare BM25-only, semantic-only, and hybrid retrieval?

I would use questions with exact fares and weights, paraphrases with few shared words, multi-section topics, ambiguous requests, and questions absent from the corpus. For each, I would label the correct source chunks and expected refusal or answer. Then I could compare recall, precision, latency, and answer quality across BM25, semantic, and hybrid modes.

RAG and answer grounding

031 After retrieval, how does the project select question-relevant passages instead of sending whole documents to the LLM?

The graph does not hand entire documents to OpenAI. It breaks retrieved passages into smaller units, scores them for overlap with the user's question, removes duplicates, and caps both the number of excerpts and total text length. That keeps the refinement prompt focused, although I would improve the selector further for paraphrases with no literal word overlap.

032 Why does the application use an LLM to refine a retrieved policy answer rather than display every retrieved passage?

Displaying every search hit produced long, distracting responses that did not answer the exact question. I use the LLM only after retrieval to turn supported excerpts into a short, direct explanation. The model is not asked to discover current policy from memory; it is constrained to the supplied evidence and must admit gaps.

033 What does 'grounded answer' mean in this implementation, and how are claims connected to citations?

Grounding means the answer's factual policy claims must be supported by retrieved source passages. The graph associates a generated claim with source chunk IDs, and the service expands those IDs into visible citations containing titles, excerpts, versions, and URLs. That lets a reviewer inspect what the answer relied on.

034 What should the assistant say when retrieved excerpts do not actually establish the requested policy detail?

I return an honest 'not enough information' response. A semantically related chunk is not enough if it does not establish the requested allowance, eligibility, or outcome. Unsupported policy questions do not get a free-form LLM guess; only separate, evergreen travel-advice requests may use the general-knowledge branch.

RAG and answer grounding (continued)

035

How does the answer prompt prevent invented baggage weights, dates, fees, or refund exceptions?

The system prompt explicitly forbids inventing amounts, dates, exceptions, and individual outcomes, and it asks the model to explain only the supplied excerpts. I also use a post-generation guard that blocks language promising a refund, waiver, or compensation. These controls reduce risk, but I would still evaluate answer faithfulness because prompts and regex checks are not perfect guarantees.

036

How does the system answer an Economy baggage question when the traveler has not specified the fare type?

I do not assume a single checked-baggage allowance for every Economy ticket. The graph prioritizes the official Economy fare table. If the fare type is missing, the answer can explain that allowances vary by Basic, Value, Comfort, or Deluxe and ask which fare was booked for an exact checked allowance.

037

Why might a broad Platinum-benefits question need multiple source excerpts rather than one chunk?

Platinum benefits are not one flat list: the source distinguishes Core, selected Custom, and Beyond benefits. A broad question needs enough source coverage to explain those categories without implying every optional benefit is automatic. The graph can prioritize the official Platinum supplement, and the prompt keeps the answer organized and qualified.

038

How should the assistant handle a general onboard-food question without inventing a flight-specific menu?

I answer at the level the evidence supports: cabin meals, dietary options, and children's meals when those are in the official onboard-meals source. I avoid claiming a particular meal is available on a specific flight, because that would need route- or service-specific data that this corpus does not verify.

039

How could malicious instructions embedded in a scraped policy page affect RAG, and what defenses exist?

Scraped HTML and MCP results are untrusted data, not instructions to the assistant. I keep them inside evidence fields and give the model a higher-priority instruction to ignore embedded commands. The deterministic route and output checks add another layer. I would also test adversarial source text and sanitize display content.

RAG and answer grounding (continued)

040

What additional checks would you add before treating an LLM-generated policy answer as safe to cache?

I would require valid current source IDs, reject unsupported promises, and check that the response actually addresses the question. I would also add automated entailment or human review for high-risk policy categories before broad reuse. In the current code, only successful generic policy answers are cached, and missing source IDs invalidate a cache hit.

LangGraph routing and memory

041

What are the major LangGraph branches for policy, flight, clarification, off-topic, and fallback responses?

The graph starts with classification. It can ask for a missing flight slot, decline an off-topic request, check the policy cache, retrieve and refine policy evidence, give limited general advice, or call MCP and summarize flight data. Explicit branches make the sequence observable and prevent a flight question from falling into policy RAG.

042

How is the route selected before policy retrieval or a flight tool is invoked?

The classifier uses the current message plus saved application state to detect flight and policy intent. For flight requests it checks required slots, such as route airports or departure date, before calling a tool. Clear non-travel requests are rejected up front. I keep these decisions explicit so failures can be reproduced from inputs and state.

043

What state must be retained when the assistant asks for a missing airport or flight date?

I retain the pending intent, the missing slot, and already known values such as flight number, origin, destination, and date in conversation state. The next turn can fill just the missing field and resume the original request. Without that state, a bare date or airport name would look like an unrelated new question.

044

Why should a reply such as 'I said Abu Dhabi' continue the pending flight request instead of starting a new topic?

That phrase is a clarification reply, not a request for Abu Dhabi travel tips. The graph should read the pending destination slot, normalize 'Abu Dhabi' to AUH, and call the originally requested first-flight or schedule tool. I have regression tests around this multi-turn behavior because losing intent at that point was a real failure mode.

045

How is recent conversation history scoped to the authenticated user and conversation?

Every conversation row has a user owner, and the service loads recent messages using both conversation ID and authenticated user ID. A user cannot supply somebody else's conversation ID to get that history. The UI's local session state improves experience, but the database ownership check is the security boundary.

LangGraph routing and memory (continued)

046

Why is chat history stored separately from the policy vector index?

Policy vectors are shared reference knowledge; personal chat turns are not. SQLite keeps recent messages and application state scoped to a user's conversation, while Chroma stores reusable policy chunks. Mixing chats into the policy index would create privacy risks and could contaminate answers for other users.

047

When should a context-dependent follow-up bypass the shared policy-answer cache?

A follow-up such as 'What about my fare?' depends on prior turns and may contain account-specific context. Reusing a globally cached answer for that text alone could be wrong or expose information across users. The cache-key logic excludes obvious personal markers, and the graph bypasses shared caching for context-dependent retrieval.

048

What happens when a policy question has no useful retrieval evidence but is not clearly off-topic?

If there is no supporting policy passage, the answer is a clear not-found statement. A narrow exception exists for evergreen travel tips that do not claim a current Etihad policy or live fact; those may use OpenAI with explicit caveats. This separates helpful general advice from unsupported policy assertions.

049

Which routing rules should stay deterministic even if an LLM could classify intent?

I keep flight intent, required slots, IATA validation, off-topic checks, cache privacy exclusions, and status normalization deterministic. They are business and safety rules where repeatability matters more than fluency. I use the LLM for natural-language refinement after trusted routing and data selection, not to override those boundaries.

050

How would you debug a user question that was incorrectly routed to RAG instead of the flight MCP tool?

I would reproduce the exact message and conversation state, then inspect the audit intent, decision, and retrieval details. Next I would check the classification patterns and pending-slot state to see why the flight branch lost priority. Finally I would add that utterance to a regression test so a later policy or prompt change cannot reintroduce the misroute.

AviationStack and MCP

051 **Why does live flight information come from AviationStack through MCP instead of from the policy corpus?**

Flight status and schedules change frequently, while the Hub corpus is a dated policy snapshot. I therefore send operational questions to AviationStack through a dedicated MCP tool instead of searching policy text. The LLM summarizes only the returned provider data, and the response identifies that source and retrieval time.

052 **Which flight tools are exposed by the MCP layer, and what is each tool responsible for?**

The MCP layer exposes flight status by number and date, airport arrivals, schedule search between airports, first-flight lookup, and route listing. These tools have typed arguments and call the AviationStack provider. The graph chooses a tool from user intent, rather than giving the model unrestricted access to every backend operation.

053 **How is the first flight from Delhi to Abu Dhabi chosen from provider records for a given date?**

I normalize the city names to DEL and AUH, choose the requested date or the local default, then request the matching route. The provider filters records to that route and date and sorts by scheduled departure to select the earliest returned flight. If the provider lacks a qualifying record, I report that limitation instead of inventing a departure.

054 **How are city names, airport aliases, and IATA codes converted into tool parameters?**

The graph maps known names and aliases such as Delhi, Abu Dhabi, and Heathrow to IATA codes, and it saves missing route slots during clarifications. The provider validates three-letter codes before sending a request. For ambiguous cities, such as London with several airports, the answer states the actual airport assumption rather than hiding it.

055 **How are AviationStack's recognized flight-status values preserved in the user-facing answer?**

I normalize the provider status and accept only the recognized values: scheduled, active, landed, cancelled, incident, or diverted. The response shows the exact status for the requested flight and date, not a vague model paraphrase. An unknown or absent status becomes an explicit not-found response rather than a fabricated status.

AviationStack and MCP (continued)

056 **Why must a future scheduled flight not be described as live operational status?**

A future flight can have a confirmed schedule without having a live operational status. The provider uses a future-schedule endpoint and labels that source separately. I make the answer say 'future schedule, not live status' so a traveler does not treat a scheduled record as confirmation that the aircraft is currently operating as planned.

057 **How does an airport-arrivals request reach the arrival-board tool rather than policy RAG?**

The classifier recognizes an arrivals-board intent before policy retrieval and extracts the destination airport. If it cannot identify the airport, it asks for the city or IATA code. Otherwise it calls the arrivals MCP tool, which filters AviationStack timetable records for that airport and returns a bounded list with a freshness caveat.

058 **What should the assistant report when AviationStack returns an empty result, an API error, or a plan-restricted endpoint?**

I distinguish 'no matching record' from 'the provider could not be reached.' Empty valid results become a not-found answer; authentication, rate-limit, network, or plan restrictions become tool-unavailable information. The application never falls through to an LLM-generated flight guess or a random policy snippet after a provider failure.

059 **Why is synthetic flight data prohibited in this project, including during provider outages?**

Synthetic operational data would look plausible but could mislead a traveler about departure times, arrivals, or cancellations. In the live app, I use only AviationStack records and say when they are missing. Test fixtures are different: they are isolated substitutes used to verify code behavior, never presented as live production flight results.

060 **How would you test date filtering, route matching, sorting, and MCP result normalization without making live API calls?**

I use a fake provider or intercepted HTTP responses with known timestamps, routes, and status values. Then I assert that the code rejects mismatched dates, orders departures correctly, normalizes MCP output, and handles empty and error cases. This gives repeatable tests without depending on a paid plan, network reliability, or a changing timetable.

Redis, SQLite, and cache design

061

What information is included in a policy-answer cache key, and why is the raw question normalized?

The key hashes the normalized generic question together with the corpus digest, chat-model name, and policy prompt/retrieval version. Normalizing case, whitespace, and trailing punctuation lets equivalent questions share an answer. Including source and version information prevents a changed policy or answering workflow from blindly reusing an older response.

062

Which policy questions are intentionally excluded from the shared Redis cache, and why?

I exclude obvious account-specific input, including email addresses, personal references, passport or PNR language, and booking identifiers. A context-dependent follow-up also should not use the shared cache. Those heuristics reduce cross-user leakage, but I would strengthen PII detection before wider deployment because regex alone cannot recognize every sensitive case.

063

How does a Redis cache hit change the number of embedding and chat-model calls?

The graph checks Redis before retrieval. On a valid hit, it reconstructs the answer and source citations from current chunk IDs and returns immediately. That means no new embedding query and no ChatOpenAI refinement call for that question. A miss can use both retrieval and the model, after which a successful generic answer may be stored.

064

What happens when Redis is unavailable, and why does the project avoid a SQLite cache fallback?

The app still answers the question, but it does not cache the result. It shows an unavailable-cache warning and periodically retries Redis. I intentionally do not write shared policy answers to SQLite because the requirement was a Redis-only shared cache; SQLite remains for users, chat history, feedback, and usage records.

065

What is the purpose of a cache time-to-live, and how would you choose one for changeable airline policies?

The current default is seven days, which limits indefinite reuse. For changeable policies I would choose the TTL together with automated corpus refresh and risk level; a short TTL is safer for fees or eligibility. The content digest and versioned key provide additional invalidation, but a stale corpus can still yield a stale new answer.

Redis, SQLite, and cache design (continued)

066 **Why does a remote Redis Cloud connection require TLS in the default configuration?**

TLS encrypts the password and cached content in transit to Redis Cloud. The default configuration rejects a remote plaintext redis:// URL and asks for rediss:// instead. A local development Redis exception is allowed. I would also rotate exposed credentials and keep the connection string out of logs, code, and committed files.

067 **How do a corpus change, model change, or prompt/retrieval version change invalidate old answers?**

Changing the corpus digest or chat-model name changes the hashed Redis key; changing the policy prompt/retrieval version does the same. Chroma independently keys its vector collection by corpus and embedding model. A subtle limit is that an embedding-model change alone is not currently in the Redis key, so I would bump retrieval version or include it explicitly when changing that model.

068 **Which data belongs in SQLite, which belongs in Chroma, and which belongs in Redis?**

SQLite owns users, conversations, messages, feedback, and LLM usage for the local MVP. Chroma owns policy embeddings and retrieval metadata. Redis owns temporary, shared generic policy answers and hit counters. Keeping these responsibilities separate prevents chat memory from becoming policy evidence and prevents the cache from being mistaken for the system of record.

069 **How would you prevent personally identifiable booking information from being shared through a cross-user cache?**

The current cache key excludes common personal markers and skips context-dependent follow-ups, while conversation history is isolated by user ID. I would strengthen that with broader PII classification, explicit no-cache rules for booking or account workflows, and privacy tests. I would never cache a response that includes personal booking details for reuse by other users.

070 **How might you prevent many simultaneous users from triggering duplicate LLM calls for the same uncached question?**

That is a cache-stampede problem, and the MVP does not yet solve it fully. I would use a short-lived Redis lock or single-flight mechanism per cache key: one worker computes the answer, and others wait briefly and re-read it. I would add timeouts so a crashed worker does not block the key indefinitely.

FastAPI, authentication, and RBAC

071

How does a user obtain a JWT, and how is that token checked on protected API endpoints?

The login endpoint verifies the username and password and returns a signed JWT with user ID, role, and expiration. Protected endpoints read the bearer token, verify its signature and expiry, then load the current user from SQLite. Looking up the user prevents a token for a deleted account from remaining sufficient on its own.

072

How are the user and admin roles enforced on backend routes rather than only hidden in the UI?

FastAPI's require_roles dependency checks the authenticated principal on the server. Both user and admin can chat and send feedback; admin-only endpoints require the admin role. Streamlit hides the dashboard for ordinary users, but that is only a presentation choice. The backend returns 403 if they call an admin URL directly.

073

Which API endpoints are public, user-accessible, or admin-only in the current application?

The public endpoints are login and health. Authenticated user and admin endpoints include /me, /chat, and /feedback. /admin/audit-logs, /admin/llm-calls, and /admin/dashboard require admin. I like describing the contract this way because it shows that observability is available without exposing user-level or provider-level details to everyone.

074

Why must the backend validate conversation ownership on every history or feedback operation?

The service checks the authenticated user when loading or creating a conversation, so another user's conversation ID cannot be used to read its state. Feedback also checks that the answer belongs to the caller, except for an explicit admin allowance. These server-side checks matter even if the UI never presents another user's identifiers.

075

How are passwords stored, and why would plaintext passwords be unacceptable?

The database stores password hashes using pwdlib's recommended password-hashing implementation rather than plaintext. That means a database leak does not directly reveal passwords, although weak demo credentials must still be changed. For a public system I would add stronger account lifecycle controls, password policy, lockout or throttling, and proper identity-provider integration.

FastAPI, authentication, and RBAC (continued)

076 **What input validation is needed for chat messages, airport codes, dates, and dashboard limits?**

Pydantic constrains chat length and feedback values; flight schemas constrain dates and IATA formats, and provider methods normalize their arguments. Admin limits are clamped server-side. I would additionally test malformed Unicode, extremely long conversations, unexpected airport aliases, and invalid dates so no request reaches a tool with unchecked parameters.

077 **Where should OpenAI, AviationStack, LangSmith, Redis, and JWT secrets be configured?**

I configure them through environment variables or a local .env that is not committed. The client UI should never receive provider secrets or the JWT signing key. For a deployed system, I would move them into a managed secret store with rotation and least-privilege access, and I would replace the development-only JWT default before sharing the app.

078 **How would you prevent an ordinary user from requesting the admin dashboard directly via HTTP?**

I enforce require_roles(Role.admin) on the admin endpoints, so a direct HTTP request with a user token is rejected with 403. I would test that with the API test client, not just click through Streamlit. Hiding the admin control in the UI is useful for clarity but cannot be the security boundary.

079 **What should the API return when a flight provider is unavailable versus when a policy answer is not found?**

The HTTP request can succeed while the assistant's domain decision is not_found or tool_unavailable. Those are distinct structured response decisions: not_found means no verified matching information, while tool_unavailable means a dependency failed or access was denied. Authentication and validation errors use appropriate HTTP errors. That distinction lets the UI explain the issue honestly.

080 **What would you change in authentication and deployment before exposing this MVP publicly?**

I would remove demo credentials, use a managed identity provider, protect secrets, add rate limits and abuse controls, tighten CORS, and deploy with monitored HTTPS. I would also revisit local SQLite and Chroma persistence, backups, policy freshness, privacy retention, and incident response. Those are future hardening steps; the current implementation is explicitly a local MVP.

UI and observability

081

How does the Streamlit interface present answers, citations, clarifying questions, and feedback?

Streamlit displays the chat turn, the assistant's decision, and expandable source details with titles and links. It can show a clarification question and accepts feedback on answers. The sidebar supports a new conversation and sign-out. I designed that around the user's immediate question while leaving evidence available for review.

082

Which metrics does the admin dashboard expose for LLM usage, cache activity, and application behavior?

The dashboard shows attempted and successful LLM calls, failed calls, token totals, latency, recent usage, and breakdowns by model or graph node. It also shows user and conversation activity, feedback, cache entries and hits, RAG mode, indexing activity, and audit events. It is admin-only because those records expose operational details.

083

What is the difference between a LangSmith trace, a SQLite LLM-usage row, and a JSONL audit event?

LangSmith traces the graph and model execution path when enabled. SQLite stores normalized per-call usage such as model, tokens, latency, status, and error type for the admin dashboard. JSONL audit events summarize turns, decisions, source IDs, and timing without raw message text. They complement one another; none is a substitute for policy provenance.

084

How would you calculate the cache hit rate and the number of avoided chat-model calls?

I would define hit rate as cache hits divided by total cache lookups, excluding requests that were intentionally not cacheable. The current dashboard exposes hits and current entries, but entries are not misses, so they cannot produce an accurate hit rate alone. I would add lookup and miss counters, then estimate avoided chat calls from eligible hits.

UI and observability (continued)

085 **Which token counts, latency values, and error fields are useful when diagnosing model cost or failures?**

Input and output tokens explain usage volume; cached-input and reasoning tokens can expose pricing or behavior differences when the provider reports them. Latency and error type show slow or failing paths. I log the graph node and model so I can distinguish policy refinement from flight summaries rather than assuming every model call has the same purpose.

086 **Why should raw user messages be minimized or omitted from operational audit logs?**

Operational logs are often exported or retained longer than chat content, so raw questions could expose names, booking references, or travel plans. The JSONL audit log records IDs, intent, decision, citations, and latency instead. SQLite still stores conversation text for context, so it needs its own access control and retention policy.

087 **How would you trace a wrong answer back through routing, retrieval, source passages, and LLM refinement?**

I would start with the response trace ID and inspect the audit route and decision. Then I would check retrieved source IDs, the selected evidence text, any cache hit, and the logged model call or MCP tool result. That tells me whether the fault came from routing, stale source content, retrieval, summarization, or cache reuse.

088 **How can the UI make it obvious whether an answer comes from a policy source or a live flight provider?**

The response includes citation source types and titles, so policy passages and AviationStack tool data can be presented differently. I would make the distinction explicit in the UI with labels such as 'Policy source' and 'Live flight data,' plus retrieval time for flight information. That avoids implying a static policy excerpt is a current flight feed.

089 **What should the health endpoint report, and what should it avoid disclosing to anonymous callers?**

The public health endpoint currently reports status, provider mode, and configuration booleans such as whether OpenAI is configured. It does not reveal secret values. Before public exposure, I would consider reducing the detail further and separating a minimal public liveness check from an authenticated readiness check for dependencies.

UI and observability (continued)

090

How would you assess whether the redesigned Etihad-style UI improves usability without confusing users about official status?

I would use short usability sessions with travelers and agents: ask them to find a policy, check a flight, inspect sources, and recover from a clarification. I would measure task completion, errors, and trust in citations. I would also make clear that the interface is an Etihad-style local assistant, not an official Etihad service or booking system.

Guardrails, tests, and next steps

091

How does the assistant politely reject off-topic requests without rejecting valid questions about meals or loyalty benefits?

The off-topic rule looks for travel vocabulary and explicit unrelated requests, while known terms include food, meals, Platinum, and loyalty. It also allows contextual follow-ups after travel conversation. That corrected the earlier over-refusal of legitimate travel questions. I would keep regression examples for both valid travel queries and genuinely unrelated requests.

092

How does the project avoid promising a refund, waiver, or compensation that written policy does not guarantee?

The policy prompt requires conditional, source-grounded language and forbids individualized promises. After generation, a deterministic check blocks text that says a refund, waiver, or compensation is guaranteed or already approved without evidence. The response instead says written policy is insufficient and directs the user to Etihad for eligibility confirmation.

093

What regression test proves semantic retrieval still runs when BM25 has no match?

The Chroma regression test asks a paraphrased policy question for which BM25 returns no candidate. A fake embedding maps it to a real pet-policy chunk, and the test verifies Chroma was queried and returned a candidate. A companion test verifies that a vector failure falls back to BM25 rather than fabricating an answer.

094

What tests cover flight clarification across turns, especially a missing date or destination airport?

I test a flight-status request followed by a bare date, and a route request followed by a clarification such as 'I said Abu Dhabi.' The expected result is that the saved intent and known slots survive and the correct MCP tool is called. I also test that genuinely missing information still produces a precise clarification question.

095

How can fake embeddings, fake Redis, and stubbed AviationStack responses keep tests deterministic?

Fakes let me control the exact input and expected output. Fake embeddings exercise Chroma ranking without OpenAI cost; fake Redis verifies shared cache hits, TTL, and outage behavior; provider stubs verify flight filtering without live API dependency. None of those fixtures are used as live data in the running application.

Guardrails, tests, and next steps (continued)

096 **Which test cases would catch a cached answer whose citations are no longer present in the current corpus?**

On cache lookup, I would remove or ignore entries whose cited chunk IDs no longer exist or are no longer current, and force retrieval again. The code already checks source IDs before serving a cached draft. Tests should cover a deleted source, a changed corpus digest, a changed model or prompt version, and an answer that should not be cacheable.

097 **How would you measure retrieval relevance and answer faithfulness using a project-specific evaluation set?**

I would create a labeled set of representative user questions with expected source chunks and answer outcomes. Retrieval metrics would include recall at k and irrelevant-hit rate; answer metrics would check factual support, citation correctness, and honest refusals. I would compare versions on the same set before tuning weights, prompts, or thresholds.

098 **What are the most important remaining limitations of the current local MVP?**

The biggest limits are that policy content is a snapshot, semantic relevance is not fully calibrated, and live flight coverage depends on AviationStack plan and availability. Redis and Chroma are local-MVP choices, and the interface is not a booking system. Some privacy and deployment controls need strengthening before public use.

099 **Which improvement would you prioritize next: better source freshness, retrieval evaluation, flight coverage, or stronger security - and why?**

I would prioritize automated source freshness first, because even perfect retrieval cannot correct an outdated policy. I would add change detection, effective-date review, and a controlled refresh process, then use an evaluation set to tune hybrid retrieval. If the client instead needs broader flight coverage, I would confirm provider access and plan limitations before promising it.

Final client pitch

100

In a two-minute interview answer, how would you explain this project's impact and its main engineering trade-offs?

I built a local travel assistant that separates two very different information needs: written Etihad policy and live flight operations. For policy questions, it checks a shared Redis cache, then combines BM25 with Chroma semantic search, selects supporting excerpts, and uses OpenAI to produce a short cited answer. For flight questions, LangGraph routes to AviationStack through MCP, preserves clarification context, and reports only returned provider facts. For example, an Economy baggage question is answered from fare-specific policy evidence, while a missing date for flight EY 219 is carried into the next turn before the flight tool runs. FastAPI enforces user and admin roles, Streamlit presents the experience, and SQLite plus optional LangSmith tracing make usage auditable. The key trade-off is honesty over apparent coverage: I would rather report missing evidence or an unavailable provider than invent a policy or flight. This is a local MVP, so my next focus would be source freshness, retrieval evaluation, and stronger deployment security.

THREE POINTS TO LAND WITH THE CLIENT

VALUE	One chat for cited policy guidance and provider-backed flight information.
CONTROL	Deterministic routing, explicit sources, and no fabricated flight results.
BOUNDARY	Local MVP today; source freshness, evaluation, and security come next.